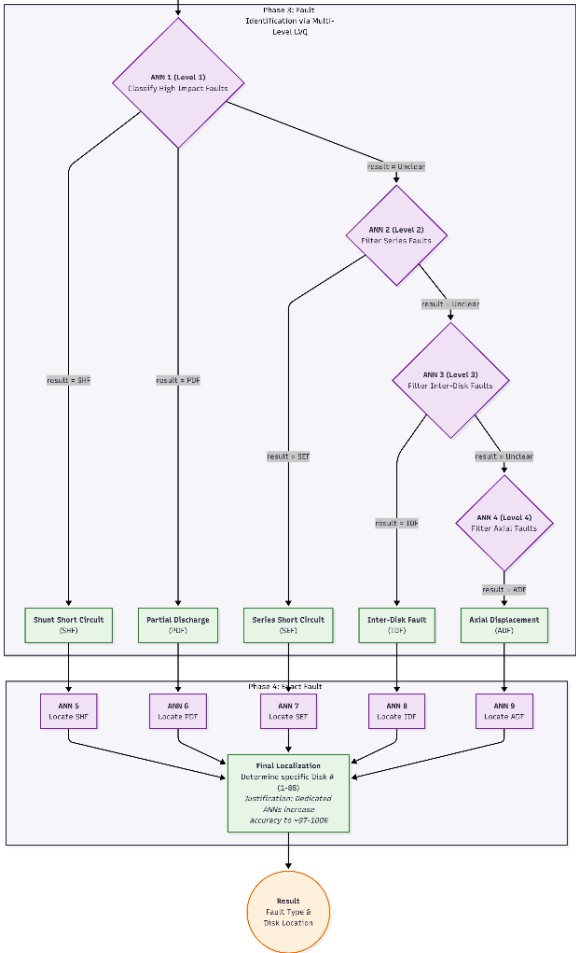
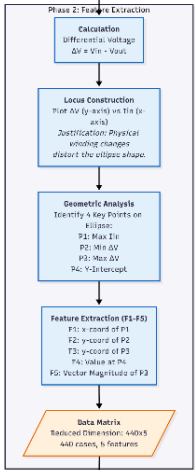
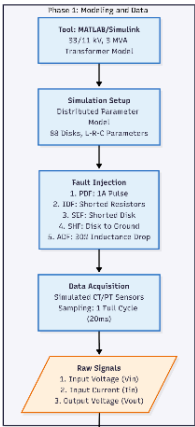




CT saturation and magnetizing (inrush) current

1) Definitions:

- **Magnetizing (inrush) current:** the very large transient current that can flow the moment a transformer is energized. It's caused by the transformer core's magnetic flux being out of balance (often because of *residual flux*), so the core briefly drives into nonlinear region (saturation) and the primary draws a big current until the flux settles.
- **CT saturation:** when a current transformer (CT) core is driven beyond its linear magnetization region so it can't faithfully reproduce the primary current on the secondary. Saturation causes waveform distortion, DC offsets, and mis-measurement — which can make a differential relay think an internal fault happened.

Those two often team up: a big magnetizing inrush plus CTs that aren't ideal = false differential trips.

2) The physics

Transformer flux and volt-second balance

The basic relation between applied voltage ($v(t)$), flux linkage ($\lambda(t)$) and turns (N) for the primary winding (ignoring resistive drop for the moment) is:

$$v(t) = N \frac{d\lambda(t)}{dt}$$

Integrate once (volt-second balance):

$$\phi(t) = 1/N \int v(t) dt + \phi_0$$

where ϕ_0 is residual flux at the instant of energization.

$$\text{If } v(t) = V_m \sin(\omega t + \theta),$$

integrating gives a sinusoidal flux plus an offset determined by ϕ_0 . If ϕ_0 positions the resultant flux excursion near or beyond the core's knee point (saturation), the core goes nonlinear and the magnetizing current skyrockets.

Magnetizing current vs flux

Magnetizing current (i_m) is set by the core magnetization curve (B–H or flux–ampere-turn relation). In the linear region:

$$\phi \propto i_m \Rightarrow i_m \approx \phi/L_m.$$

But when the core saturates, a small increase in flux requires large magnetizing ampere-turns → i.e., i_m shoots up nonlinearly.

Residual flux and inception angle matter

- The **residual flux** (remanent flux after the transformer was last de-energized) can add or subtract to the flux that would be produced by the applied voltage. If it *adds* such that the peak flux exceeds the knee for the first half-cycle, a massive inrush occurs.
- The **voltage phase angle at switching (inception angle)** controls how the sinusoidal integral (volt-seconds) adds with residual flux — certain angles produce much worse inrush.

CT magnetization and saturation

A CT is a transformer too. Its secondary current (i_s) ideally equals primary current (i_p) scaled by the turns ratio. But the CT core has finite magnetizing inductance L_m^{CT} and nonlinear magnetization curve. When the CT flux tries to exceed the CT's knee flux (because the secondary sees a waveform with heavy DC offset or huge differential ampere-turns), the CT will saturate and the secondary current waveform no longer scales linearly — it becomes clipped/distorted.

3) Waveform signatures

Magnetizing inrush

- **Large amplitude current** at the instant of energization, often several times rated current (2–10× or more, depending on conditions).
- **Transient DC offset / half-cycle asymmetry** in the waveform — not a perfect sinusoid.
- **High second-harmonic content** (and other low-order even/odd harmonics) because saturation and asymmetry distort the waveform. Traditionally, second-harmonic energy ratio (2nd / fundamental) is used to detect inrush.
- **Decaying envelope**: inrush amplitude decays over several cycles (tens to hundreds of ms) as flux damps and core returns to linear operation.

Sketch (text): first few cycles: massive spike and asymmetric baseline; after ~5–30 cycles it settles to normal magnetizing current.

CT saturation (as measured on relay side)

- **Abrupt clipping or flattening** of secondary waveform once CT saturates.
- **Secondary waveform lags or is missing part of cycle**, producing differential current when compared across transformer zones.
- **Harmonic content**: saturation introduces broadband high-frequency energy and distortion; unlike inrush, CT saturation may introduce sharp corners and high-frequency components.
- **False differential spikes**: when CT saturates on one side and not the other, the relay sees large imbalance.

4) Distinguishing features (inrush vs internal fault vs CT-sat)

Feature	Internal fault	Magnetizing inrush	CT saturation artifact
Amplitude	Large, sustained	Large, decaying	Can be clipped/abrupt

Harmonic content	Less 2nd-harmonic, stronger odd harmonics and broadband	High 2nd-harmonic (classic)	Broadband high-freq + distortion
Phase relation across phases	Often asymmetric but follows fault location	Usually centered on energization instant and correlated to residual flux	Appears when CT burden/conditions cause nonlinearity, may be one-sided
Time behavior	Persists until cleared	Decays over cycles	Can persist while CT is saturated (depends on primary waveform)

Important nuance: **second-harmonic test is helpful but not foolproof**. Some internal faults may also produce second-harmonic energy; conversely, CT saturation can reduce harmonic content making an inrush look like a fault. Hence many modern methods combine time-frequency analysis, flux estimation, or ML.

5) Modeling CT saturation and inrush in simulation

CT model elements

A realistic CT model contains:

- **Primary and secondary windings** (turns ratio).
- **Magnetizing branch**: nonlinear magnetizing inductance ($L_m(i)$) or a B–H curve (hysteresis optional).
- **Leakage inductance** and **winding resistance**.
- **Saturation knee** described by a nonlinear curve — e.g., a piecewise model or using an analytic B–H curve (e.g., Jiles–Atherton or simple polynomial fits).
- **Burden**: the relay and wiring impedance on the secondary.

Transformer inrush simulation steps

1. Use an electromagnetic transient (EMT) tool like PSCAD/EMTDC or ATP-EMTP.

2. Model the power system source voltage, transformer, residual flux (set initial core flux state), and CTs with nonlinear magnetization.
3. Vary residual flux and switching angle; record primary and secondary currents.
4. Optionally include measurement noise, CT tolerance differences, and realistic burdens.

Practical model tip: residual flux

You can set initial flux state in many EMT tools. Sweep ϕ_0 from $-\phi_{\text{sat}}$ to $+\phi_{\text{sat}}$ and sampling switching angle from 0 to 360°.

6) Quantitative relations (useful equations & intuition)

Volt-second relation simplified (sinusoidal source):

$$\phi(t) = -V_m/(\omega N)\cos(\omega t + \theta) + \phi_0$$

Peak flux magnitude:

$$\phi_{\text{peak}} = |V_m/(\omega N)| + |\phi_0|.$$

If $\phi_{\text{peak}} > \phi_{\text{knee}} \rightarrow$ possible saturation and inrush.

Residual flux effect example

If nominal sinusoidal flux amplitude (without residual) is 1 pu and core knee is at 1.05 pu, a residual flux of 0.2 pu in the same direction can push the peak to 1.2 pu $\rightarrow$ deep saturation.

CT magnetization approx.

CT magnetizing current (i_m^{CT}) relates to flux via the nonlinear magnetization curve; in linear region ($i_m^{\text{CT}} \approx \phi / L_m$). In saturation, incremental inductance drops, so $\Delta\phi$ needs much larger Δi_m^{CT} .

7) Detection algorithms and features people use

Classic (relay) methods

- **Second-harmonic restraint:** measure fraction of energy in 2nd harmonic: $R_2 = E_{2\text{nd}}/E_{\text{fund}}$. If $R_2 >$ threshold (e.g., 10–20%), restrain differential trip (i.e., treat as inrush). Simple and fast, but can fail when harmonics are altered by CT saturation.

- **Half-cycle energy methods:** compare energy per half cycle; inrush tends to produce asymmetry/large half-cycle energy differences.

Modern signal analysis

- **Wavelet transform (WT/CWT/VMD):** isolates transient components — useful to see high-frequency CT saturation artifacts vs lower-frequency inrush signature.
- **Time–frequency spectrograms:** visualize harmonic growth/decay over cycles.
- **Flux estimation:** estimate core flux from measured voltage and integrate; if estimated flux > threshold, suspect inrush.
- **CT saturation detection:** detect sudden flattening/clipping or DC offsets on CT secondary; algorithms may flag saturation based on derivative changes or spectral content.

Machine learning / deep learning features

- Raw waveform windows (multi-cycle) to CNN/LSTM.
- Harmonic ratios (2nd/1st, 3rd/1st), energy in band-limited windows.
- Wavelet coefficients or VMD modes as inputs.
- Time-derivative features, phase differences across phases, envelope decay rates.
- Additional CT-specific features: sudden amplitude truncation, kurtosis, or high-frequency energy spikes.

8) Mitigation & practical fixes

Hardware-level

- **Use CTs with higher knee point / better magnetics** for critical circuits.
- **Reduce burden** (keep wiring short and use low-impedance relays) to lower CT secondary voltage needed for a given current (helps avoid saturation).

- **Proper CT rating & sizing:** match primary expected fault currents and inrush conditions.

Relay/algorithm-level

- **Adaptive differential criteria:** adjust thresholds dynamically depending on detected conditions (e.g., change restraint when inrush signature detected).
- **Pre-filtering / saturation detection blocks:** detect CT saturation and either compensate or hold tripping until compensated.
- **Flux-based checks:** use integrated voltage to compute flux and block trip during expected inrush flux excursions.
- **Compensating algorithms:** try to reconstruct true primary currents by estimating CT magnetization current and subtracting it (model-based correction).

Data & ML strategies

- **Train with CT saturation scenarios** (augment data with CT nonlinearity) so models learn to ignore CT artifacts.
 - **Multi-stage classifiers:** stage 1 detects inrush/CT saturation, stage 2 classifies internal faults only if stage 1 says “no inrush.”
 - **Explainable outputs:** relay reports confidence and dominant reason (inrush vs CT-sat) to the protection engineer.
-

9) Experimental design suggestions (lab)

Lab bench

- Use a small single-phase transformer, controlled voltage source, and an oscilloscope sampling at high rate.

- Pre-magnetize core (inject DC to make residual flux) to emulate residual flux then switch at controlled angle.
 - Measure inrush and CT behavior with different burdens.
-

10) Why this matters for differential protection design

- **False trips** remove a transformer from service unnecessarily (costly, risky). Differential relays must distinguish inrush from internal fault with near-perfect reliability.
 - **Missed genuine faults** are even worse — so a conservative protection system must balance sensitivity and restraint.
 - CT saturation and inrush are two main causes of wrong decisions; understanding their physics and signatures is essential for safe, ML-assisted protection design.
-

11) Practical checklist for researchers / engineers

- When simulating inrush, always set **initial flux** and sweep **switching angle**.
 - Include **nonlinear CT models** and realistic burdens when evaluating differential logic.
 - Test algorithms on **worst-case residual flux** plus weak-source conditions.
 - Use **time–frequency** features (wavelet/VMD) *and* harmonic ratios — they are complementary.
 - If using ML, **augment** training data with CT-saturation examples and clearly report worst-case false-trip rates (not just average accuracy).
 - Consider combining **physics-based (flux) checks** with learned models for safety.
-

1) Implementation-ready blueprint

A. Keeping the classical 87T core (safety backbone)

We do **not** replace the differential relay, rather we **augment** it.

Core blocks (already standard in a numerical relay):

1. **ADC + anti-alias + resampling**
2. **CT ratio & polarity correction**
3. **Vector group compensation** (phase shift and scaling HV ↔ LV)
4. **Idiff and Irest** calculation

5. **Bias characteristic** (slope1/slope2 + breakpoint)
 6. **High-set Idiff** element (fast internal fault trip)
 7. **Security features** (stabilization during external faults, CT saturation suspicion, etc.)
-

B. The paper's method as a "supervisory classifier" (the brain that decides *what kind of event it is*)

This supervisor only runs when something happens.

Trigger (disturbance start):

- Start supervisor when **|Idiff|** rises above a small arming threshold (not the trip threshold). This is the "something started" moment.

Supervisor pipeline:

1. Take a short window of current samples (more on window size below)
 2. Compute wavelet decomposition (db4 in the paper)
 3. Convert coefficients into a compact set of "energy" features (EV)
 4. Feed feature sequence to the LSTM
 5. Output class + confidence:
 - **INRUSH**
 - **INTERNAL FAULT/DEFECT**
 - **EXTERNAL FAULT / CT SATURATION**
-

C. Trip logic (how we combine classic 87T with ML safely)

State machine (implementation-ready)

NORMAL

- Classic 87T runs, but no trip unless we're firmly in operate region.

DISTURBANCE_DETECTED

- Latch disturbance if **|Idiff|** exceeds arming threshold.
- Start supervisor window.

CLASSIFY

- Run ML supervisor each window.
- Require stability: e.g., “same class for 2 or 3 consecutive windows”.

DECISION

- If (Class = INTERNAL_FAULT) AND (87T operate condition true) → **TRIP**
- If (Class = INRUSH) → **BLOCK / RESTRAIN** differential element for a short time
- If (Class = EXTERNAL/CTSAT) → **FORCE RESTRAIN** + optionally enable a “CT saturation secure mode”

RESET

- When Idiff drops below a reset threshold for some time, return to NORMAL.

Why this is “relay grade”

- ML never trips by itself.
 - ML mainly decides **block vs allow trip**, and classic 87T still enforces the physics boundary.
-

2) Key engineering choices

A. Window length and timing (this must be consistent)

You must decide one of these two designs:

Option 1: Fast protection window (relay-like)

- Use $\frac{1}{4}$ cycle to $\frac{1}{2}$ cycle windows (very fast)
- Features computed on short windows
- LSTM sees a short sequence (like 4–8 subwindows)

Option 2: Slower but stronger classification

- Use 1–2 cycles for better separability
- Trip decision delayed (might be unacceptable for internal faults unless high-set element covers it)

Practical relay compromise

- High-set 87T trips fast (sub-cycle) if Idiff is huge.
 - ML supervisor handles ambiguous cases, even if it takes slightly longer.
-

B. Deterministic runtime (real-time firmware rule)

Wavelet + LSTM must run with fixed worst-case execution time.

Implementation tips

- Precompute wavelet filter coefficients.
- Use fixed-point or carefully bounded floating-point.
- Run ML only in DISTURBANCE/CLASSIFY state, not continuously.
- Quantize the LSTM model (common in embedded inference).

Produce a **trip truth table** (Mode × Operate region × confidence)

C. If ML is wrong or uncertain behavior

You need explicit fallback rules:

- If confidence < threshold → ignore ML and use classic logic
- If ML output is invalid → ignore ML
- If ML contradicts high-set internal fault → allow trip (high-set wins)
- Provide a setting: “ML Supervisory = OFF / BLOCK-only / FULL-supervisory”

This is how you make it acceptable to protection engineers.

D. CT saturation handling

Implemented a clear rule:

- Detect possible CT saturation (any of the standard approaches: waveform distortion indicators, derivative-based flags, or ratio checks)
- If saturation suspected:
 - Increase restraint temporarily
 - Require more consecutive ML confirmations before trip
 - Or temporarily block 87T unless high-set element asserts strongly

This turns “robustness” into a deterministic engineered feature.

3) Settings list

Classic 87T settings

- Slope1, slope2, breakpoint
- High-set Idiff and delay
- Minimum pickup Idiff
- Harmonic restraint enable/disable (optional fallback)

ML supervisor settings

- Enable: Off / Block-only / Supervisory
 - Confidence threshold
 - Consecutive-window count (e.g., 2 windows)
 - Disturbance arming threshold
 - Class-specific hold times (how long you block after inrush classification)
 - Event recording enable (store features and class output)
-

4) Verification plan

You need three layers of testing.

Layer 1: Algorithm validation (offline)

- Confusion matrix for classes
- Stress tests: noise, frequency drift, phase-angle variation, remanent flux, different switching angles

Layer 2: Real-time / HIL testing

- Run on target processor or close equivalent
- Confirm worst-case execution time
- Confirm decisions match offline results

Layer 3: Protection-grade security tests

- External faults + CT saturation (must not trip)
- Inrush with sympathetic inrush, recovery inrush, overexcitation
- Internal faults under low current (sensitive region)
- Mixed cases: inrush + internal fault inception during energization (nasty but real)

Success criterion is not “accuracy”. It’s:

- **Dependability** (trips when it must)
 - **Security** (doesn’t trip when it must not)
 - **Speed** (within acceptable clearing time)
 - **Determinism** (bounded runtime)
 - **Auditability** (records explain why it acted)
-

5) Advantages and uniqueness over traditional designs

Traditional 87T inrush discrimination

Usually uses:

- 2nd harmonic restraint/block
- Sometimes 5th harmonic for overexcitation
- Waveform-based rules, threshold tuning, and filtering

The pain points

- Modern transformers and system conditions can reduce harmonic content
- CT saturation and noise can distort harmonic estimates
- Some internal faults can also show harmonics, confusing classic logic

What’s uniquely better in the paper’s approach

1) Uses time-frequency “shape” not just harmonics

Wavelets capture *how the waveform changes over time*, not only “how much 2nd harmonic exists”.

That can help when harmonics are unreliable or weak.

2) Learns sequential patterns (LSTM)

Inrush has a characteristic evolution (decay, asymmetry patterns), and internal faults have a different evolution.

LSTM is built to recognize that kind of “story over time”.

3) Potentially better security during CT saturation

External faults with CT saturation can create fake differential current. A classifier trained on those patterns can act as an extra “do-not-trip” guard.

4) More adaptive across conditions (if trained properly)

Instead of hand-tuning many thresholds for every transformer fleet, you can retrain the model using real disturbance records.

6) The honest tradeoffs

- **More complexity:** wavelet + ML increases development and validation burden
 - **Needs good training coverage:** if the model never saw a scenario, it may misclassify
 - **Explainability:** we must provide event logs that show: I_{diff}/I_{rest} + class + confidence + state transitions
 - **Certification mindset:** utilities and standards folks will demand a conservative fallback mode
-

Implementation-ready summary (one paragraph)

Build a conventional biased transformer differential relay with high-set and stabilization, then add a disturbance-triggered wavelet-energy + LSTM supervisor that classifies inrush vs internal fault vs external/CT saturation. Use a state machine with confidence thresholds and consecutive-window confirmation. Let ML mainly control blocking/restraint, while classical 87T logic remains the final authority for tripping (except high-set which always wins). Provide settings, deterministic runtime, fallback rules, and a rigorous security-focused test plan.

Perfect. Let's "wire" the Springer-style **adaptive differential protection** into the same chassis as an **ML-supervised 87T**, so it becomes a single coherent hybrid relay (not two ideas stapled together).

I'll do two things:

1. **Direct mapping**: Springer adaptive algorithm blocks/changes what, ML supervises what
 2. A **hybrid relay architecture diagram** you can paste into a report (ASCII + Mermaid)
-

1) Direct mapping: Springer adaptive algorithm ↔ ML-supervised 87T

What Springer "adaptive" really is (in protection terms)

The Springer chapter's adaptive idea (as typically used in transformer 87T research) is essentially:

- Keep the 87T operate/restraint structure
- Detect "nasty non-fault distortions" (especially CT saturation, inrush-like behavior)
- Adapt the relay characteristic in real time:
 - increase restraint (raise slope, raise pickup, add delay) when security risk is high
 - restore sensitivity when security risk is low

So the Springer block outputs a **Protection Mode** and **Adaptive Settings**.

What ML-supervised 87T is

The ML layer outputs a **Class** and **Confidence** like:

- INRUSH
- INTERNAL_FAULT
- EXTERNAL_FAULT/CT_SAT
- (Optional: OVEREXCITATION)

Then it either:

- Blocks the 87T,
- Permits it,
- Or tightens/loosens restraint.

The clean mapping (how they become one design)

Think of them as two advisors to the same “Operate/Restrained” engine:

- Springer adaptive logic is a **physics-rule-based advisor**
- ML is a **pattern-recognition advisor**
- The 87T bias characteristic is the **final authority** (except high-set)

Mapping table

Hybrid function	Springer adaptive piece (rule-based)	ML-supervised piece
Detect CT saturation risk	Derivative/flat-top/flux indicator, mismatch checks	“EXTERNAL/CTSAT” class from learned patterns
Handle CT saturation	Increase slope2, add operate delay, require confirmation	Force RESTRAIN / increase confirmation count K
Detect inrush	Harmonic ratio / waveform indicators / energization cues	“INRUSH” class
Handle inrush	Block/Restrained 87T for hold time, adaptive pickup	Block 87T + maintain block until confidence drops
Internal fault decision	Standard 87T operate region + high-set	“INTERNAL” class acts as permission/accelerator
Ambiguous cases	Conservative bias or delay	Confidence score chooses conservative vs fast path
Fail-safe	Always exists (rules deterministic)	If ML uncertain → ignore ML, fall back to adaptive+classic

2) The hybrid decision logic (relay-grade)

Core principle

ML never trips alone.

It can only *permit* trip or *block* trip, while classic 87T still checks operate/restraint.

Key signals

- $(I_{diff}), (I_{rest})$
- High-set assertion: $(I_{diff} > I_{HS})$

- Springer indicators:
 - CT_SAT_FLAG (rule-based)
 - INRUSH_FLAG (rule-based, optional)
- ML outputs:
 - CLASS $\in \{\text{INRUSH, INTERNAL, EXT/CTSAT}\}$
 - CONF $\in [0,1]$

Trip permission logic (practical)

1. If **High-set** $\rightarrow$ TRIP immediately.
2. Else:
 - If ML says INRUSH with high confidence $\rightarrow$ BLOCK
 - Else if ML says EXT/CTSAT with high confidence OR CT_SAT_FLAG is true $\rightarrow$ RESTRAIN (tight bias)
 - Else if ML says INTERNAL with high confidence AND 87T operate $\rightarrow$ TRIP (after K confirmations)
 - Else $\rightarrow$ classic adaptive 87T (Springer) decides conservatively

What adaptivity actually changes in the characteristic

Your adaptive module modifies parameters like:

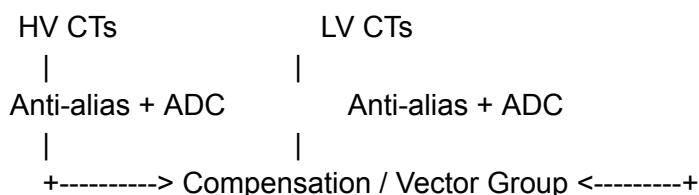
- Pickup: (I_{pickup})
- Slope1, Slope2
- Breakpoint (I_{brk})
- Operate delay (tiny security delay)
- Confirmation requirement K

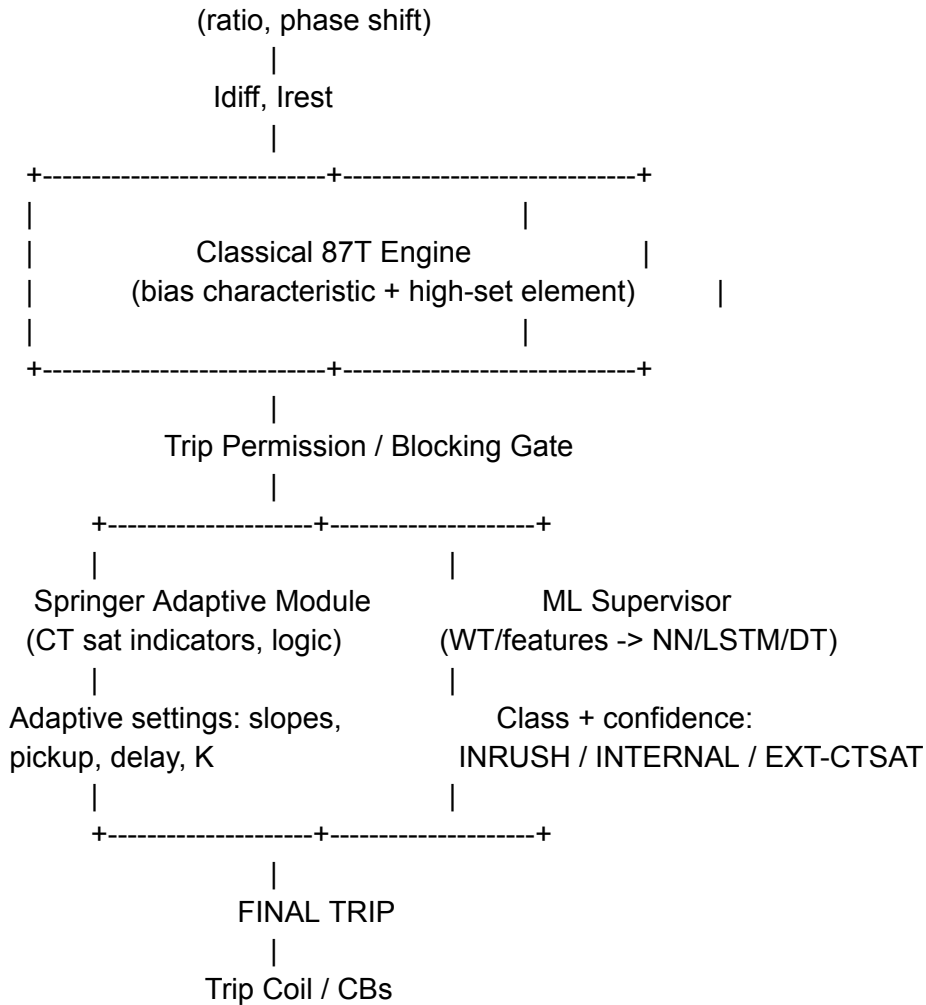
Example policy:

- NORMAL: slopes = (S1,S2), K=2
- CT_SAT mode: S2 $\uparrow$, pickup $\uparrow$ slightly, K=3 or 4
- INRUSH mode: block for hold time, K irrelevant because trip disabled

3) Hybrid relay architecture diagram

3.1 Simple “relay block” diagram (ASCII)





3.2 Mermaid diagram (clean for thesis/report)

Paste this into Mermaid:

flowchart TD

A[HV CTs] --> B[Analog Front-End + ADC]

C[LV CTs] --> D[Analog Front-End + ADC]

B --> E[Compensation\nRatio + Vector Group + Phase Align]

D --> E

E --> F[Compute Idiff & Irest]

F --> G[Classical 87T Core\nBias Characteristic + High-Set]

G --> H[Trip/Block Gate]

%% Adaptive module (Springer-style)
F --> I[Adaptive Logic (Springer)\nCT Sat Detect + Adaptive Bias\nAdjust:
Slope1/Slope2/Pickup/Delay/K]
I --> H

%% ML supervisor
F --> J[ML Supervisor\nFeature Extraction (e.g., WT/EV)\nNN/DT/LSTM Classifier]
J --> K[Class + Confidence\nINRUSH / INTERNAL / EXT-CTSAT]
K --> H

H --> L[Trip Output Logic]
L --> M[Trip Coil -> Circuit Breakers]

4) The “hybrid state machine” (how it behaves over time)

Recommended states

- NORMAL
- DISTURBANCE_DETECTED
- CLASSIFY
- MODE_SELECTED (INRUSH / CT_SAT_SECURE / NORMAL_FAULT_EVAL)
- TRIP or RESTRAIN/BLOCK
- RESET

Practical rules (very relay-like)

- Enter DISTURBANCE when $I_{diff} > \text{arming threshold}$
 - Run ML + adaptive indicators for N windows
 - Choose mode:
 - If INRUSH (ML high conf OR rule-based) → BLOCK
 - If CT_SAT risk high (rule OR ML) → CT_SAT_SECURE (tight bias)
 - Else → NORMAL_FAULT_EVAL
 - Trip only if:
 - High-set OR
 - (Operate region true AND INTERNAL classification stable for K windows)
-

5) Why this mapping is strong (design-review friendly)

This hybrid architecture makes both camps happy:

- Protection engineers see a **classical 87T** with proven bias logic.
 - Adaptive rules (Springer) provide **deterministic stability** under CT saturation.
 - ML adds **better discrimination** in tricky overlap cases (weak harmonics, mixed events).
 - If ML fails or is uncertain, relay still works safely.
-

If you want, I can tailor this to your specific paper/pipeline

Tell me the ML model you're using (WT-LSTM, CNN, DT), and I'll:

- Add the exact **feature vector** and **window timing**
- Produce a **trip truth table** (Mode × Operate region × confidence)
- Generate a **final diagram** with labeled signals (Idiff/Irest/2nd harmonic/CTsat flag/score) suitable for a thesis figure

Below is a **relay-grade trip truth table** you can directly drop into a design doc. It assumes the hybrid philosophy we discussed:

- **High-set (instantaneous) always wins**
 - Otherwise, **Classical 87T operate region** must be true to trip
 - ML is **supervisory**: it can **block**, **restrain**, or **permit**, but not trip alone
 - “Mode” is the *selected operating mode* (from Springer adaptive + ML classification)
 - “Confidence” is ML confidence, and “Operate region” is classical 87T characteristic (Idiff vs Irest) output
-

Definitions used in the table

Discrete inputs

- **Mode** $\in$ {NORMAL_FAULT_EVAL, INRUSH_BLOCK, CTSAT_SECURE, INTERNAL_FAULT_PERMIT}
 - **NORMAL_FAULT_EVAL**: no strong evidence of inrush/CTsat, evaluate normally (adaptive bias may still be active mildly)

- **INRUSH_BLOCK**: inrush suspected (ML and/or rule-based)
- **CTSAT_SECURE**: external fault + CT saturation suspected (ML and/or rule-based)
- **INTERNAL_FAULT_PERMIT**: ML strongly indicates internal fault pattern
- **Operate region (OP)** $\in \{0, 1\}$
 - **1** means classical biased differential says “operate” (trip region)
 - **0** means classical characteristic says restrain
- **Confidence bucket** (based on ML confidence **p**)
 - **HIGH**: $p \geq 0.85$
 - **MED**: $0.60 \leq p < 0.85$
 - **LOW/NA**: $p < 0.60$ or ML unavailable/invalid

Outputs

- **Action** $\in \{\text{TRIP, BLOCK, RESTRAIN, HOLD/WAIT}\}$
- **Notes** includes what the relay does to settings (Springer adaptive knobs)

Global override (applies before the table)

Condition	Action
High-set asserted ($I_{diff} > I_{HS}$)	TRIP immediately (ignore Mode/confidence)

Trip truth table (Mode × OP × Confidence)

1) Mode = INRUSH_BLOCK

OP	Confidence	Action	What relay should do
0	LOW/NA	BLOCK	Block 87T trip output; keep monitoring until reset
0	MED	BLOCK	Same as above
0	HIGH	BLOCK	Same as above
1	LOW/NA	BLOCK + HOLD/WAIT	Still block (security-first). Re-check next window(s)

1	MED	BLOCK + HOLD/WAIT	Block and extend inrush hold timer
1	HIGH	BLOCK	Hard block (most secure)

Adaptive setting note: raise pickup or add a small “operate inhibit timer,” keep slope conservative.

2) Mode = CTSAT_SECURE (external fault / CT saturation suspected)

OP	Confidence	Action	What relay should do
0	LOW/NA	RESTRAIN	Normal restraint, but keep CTsat flag active
0	MED	RESTRAIN	Increase slope2, increase K (confirm count)
0	HIGH	RESTRAIN	Strong restraint, log CTsat event
1	LOW/NA	HOLD/WAIT	Do not trip yet; require more evidence
1	MED	HOLD/WAIT + RESTRAIN	Tighten bias; require INTERNAL mode before permitting trip
1	HIGH	RESTRAIN (no trip)	Treat as “do-not-trip” unless high-set occurs

Adaptive setting note: slope2 ↑, pickup ↑ slightly, confirmation K ↑ (e.g., 3–4), optional short delay.

3) Mode = INTERNAL_FAULT_PERMIT

OP	Confidence	Action	What relay should do
0	LOW/NA	HOLD/WAIT	ML says internal but 87T doesn't: wait for OP or more data
0	MED	HOLD/WAIT	Same; do not trip without OP

0	HIGH	HOLD/WAIT	Still no trip (classical boundary not met). Re-evaluate quickly
1	LOW/NA	HOLD/WAIT	OP true but ML weak: require more windows or revert to NORMAL_FAULT_EVAL
1	MED	TRIP (after K)	Trip after K consecutive INTERNAL predictions (K typically 2–3)
1	HIGH	TRIP (fast, after K=1–2)	Permit faster trip (smaller K, minimal intentional delay)

Adaptive setting note: restore sensitivity (normal slopes), reduce delays, reduce K for speed when confidence is high.

4) Mode = NORMAL_FAULT_EVAL (no strong classification)

OP	Confidence	Action	What relay should do
0	LOW/NA	RESTRAIN	Normal secure state
0	MED	RESTRAIN	Normal restraint
0	HIGH	RESTRAIN	Normal restraint
1	LOW/NA	HOLD/WAIT (or TRIP with delay)	Conservative: either wait K windows or apply small delay
1	MED	TRIP (after K=2–3)	Trip after confirmation windows (slightly conservative)
1	HIGH	TRIP (after K=2)	Trip with standard confirmation

Adaptive setting note: use your “default adaptive bias” (slope1/slope2 normal), K moderate.

Recommended “K confirmation” policy (practical defaults)

You'll want K to vary with mode:

- **INRUSH_BLOCK**: K irrelevant (trip blocked)
- **CTSAT_SECURE**: K high (3–4) and/or require INTERNAL_FAULT_PERMIT before trip
- **NORMAL_FAULT_EVAL**: K = 2–3
- **INTERNAL_FAULT_PERMIT**:
 - HIGH confidence: K = 1–2
 - MED confidence: K = 2–3

This is how you get both **speed** and **security**.

How Mode should be selected (one clean rule)

You can implement Mode selection as priority logic:

1. If rule-based INRUSH flag OR ML says INRUSH with $p \geq 0.85 \rightarrow$ **INRUSH_BLOCK**
 2. Else if rule-based CTsat flag OR ML says EXT/CTSAT with $p \geq 0.85 \rightarrow$ **CTSAT_SECURE**
 3. Else if ML says INTERNAL with $p \geq 0.85 \rightarrow$ **INTERNAL_FAULT_PERMIT**
 4. Else $\rightarrow$ **NORMAL_FAULT_EVAL**
-

If you want, I can also generate:

- a **compact version** of this table for a paper (smaller)
- a **Boolean equation form** (for firmware implementation)
- a **timing diagram** showing windows, K confirmations, and trip timing

In this work, machine learning is intentionally employed in a supervisory and adaptive role rather than as a primary tripping mechanism. The classical differential protection logic remains the final authority for tripping decisions, ensuring deterministic behavior and fail-safe operation. The ML module contributes by classifying operating conditions and adjusting protection behavior—such as enabling restraint, modifying bias parameters, or permitting faster operation—thereby enhancing security and dependability without compromising protection principles.

The adaptive differential protection framework provides deterministic parameter adjustment based on identified operating conditions, while the ML classifier enhances condition

identification accuracy. Together, they form a complementary hybrid system: adaptive logic governs how the relay should respond, and ML assists in deciding which adaptive mode is appropriate.

So it may:

- Increase restraint
- Increase bias slope
- Require confirmation over more samples
- Temporarily block operation

It simply **changes its tolerance** while waiting for clarity.

An adaptive relay operates by continuously adjusting its sensitivity and restraint based on the perceived operating condition rather than relying on fixed settings. Instead of making immediate decisions during abnormal transients, the relay temporarily adopts a conservative posture, allowing waveform characteristics to evolve and reveal their nature. Benign conditions such as inrush or CT saturation are handled with increased restraint, while persistent internal faults lead to restored sensitivity and decisive tripping. This behavior mimics the reasoning of an experienced protection engineer, enhancing both security and dependability without sacrificing determinism

1. Simulation Settings

Block: **powergui**

- **Simulation Type:** **Discrete**
 - **Sample Time:** **1e-5 s**
 - **Implication:** We chose **Discrete** to mimic the "Sampling" behavior of a digital relay. We chose **1e-5 s** to get high-resolution data of the sharp Inrush spikes.
 - **Solver (Preferences):** **Backward Euler**
 - **Implication:** Provides numerical stability for the differential equations caused by the saturating transformer core.
-

2. The Grid (Primary Source)

Block: **VSa, VSb, VSc**

- **Peak Amplitude:** **1.8779e+05 V**
 - **Formula:** $230,000 * \sqrt{2}/\sqrt{3}$
 - **Implication:** This generates a **230 kV** (Line-to-Line) RMS system. The division by $\sqrt{3}$ converts Line voltage to Phase voltage, and $\sqrt{2}$ converts RMS to Peak.
 - **Phase:**
 - Phase A: **0** degrees
 - Phase B: **-120** degrees
 - Phase C: **120** degrees
 - **Frequency:** **50 Hz**
 - **Implication:** Standard grid frequency.
-

3. Circuit Breakers (Switchgear)

Block: **Three-Phase Breaker (Primary)**

- **Switching Times:** **External** (Checked)
 - **Implication:** Controlled by a logic Step block (allowing precise "Point-on-Wave" switching to generate max Inrush).

- **Breaker Resistance (**Ron**): 0.01**
 - **Implication:** Represents a healthy, low-resistance contact.
- **Snubber Resistance (**Rs**): 1e6 ohm**
 - **Implication: CRITICAL STABILITY FIX.** Creates a tiny "leakage path" when the breaker is open. This prevents the "Open Circuit" topological error that crashes Simulink.
- **Snubber Capacitance (**Cs**): inf**
 - **Implication:** Removes the capacitive snubber, leaving only resistive damping (simpler for the solver).

Block: Three-Phase Breaker1 (Secondary)

- **Same settings as Primary.**
 - **Implication:** Used to disconnect the load (time step > 10s) during Inrush training runs.

4. The Measurement System (CTs)

Block: CT 2000/5 A

- **Nominal Power: 25 VA**
- **Winding 1 Parameters: [5*1/20 0.001 0.04]**
 - **Implication:** Sets the Turn Ratio (2000:5) and the internal impedance of the primary winding.
- **Saturation Characteristic: [0 0 ; 0.01 10 ; 1 10.5]**
 - **Implication:** Defines the **Non-Linear Knee Point**.

Block: Goto

- **Tag: I1**
 - **Implication:** "Wireless" transmission of the Primary Current signal to Scope or Relay Logic subsystem to keep the diagram clean.

Block: RMS

- **Type: Running RMS**
 - **Implication:** Calculates the RMS value over a moving window (one cycle). Used for the display boxes (sanity check), NOT for the Neural Network input.

5. Stabilizers (The Fixes)

Block: Three-Phase Series RLC Branch

- **Resistance (R):** 0.1 ohm
- **Inductance (L):** 1e-3 H
 - **Implication:** These are the small impedances placed between the Main Transformer and the CTs. They serve to **Decouple** the two inductive components, helping the math solver avoid the "Series Inductor" conflict.

Block: **Three-Phase Series RLC Load**

- **Parameter: Configuration (grounded)**
 - **Implication:** Transformer secondary is **Delta (D1)** connected, which means it has no neutral point (it is "floating"). By connecting a **Grounded Wye (Y)** load, we physically tie the secondary circuit to the earth (0V).
- **Parameter: Nominal phase-to-phase voltage Vn 11e3 V** (11,000 Volts)
 - **Implication:** This matches the **Secondary Voltage** of your step-down transformer..

Power Consumption

This determines how much current flows when the breaker is closed.

- **Parameter: Active power P (W) 10e6 W**(10 Megawatts)
 - **Implication:** simulating a "Light Load" scenario.

$$I_{load} = 10e6 / \sqrt{3} * 11k = 525A$$

Since your transformer is rated for **300 MVA**, a 10 MW load is very small (approx 3% loading).

- **Parameter: Inductive reactive power QL 100** (Vars)
 - **Implication:** Effectively 0. Simulating a **Purely Resistive Load** (Unity Power Factor) for simplification

6. Fault Generators (Training Data)

Block: **Internal_Fault**

- **Fault Resistance (R_{on}):** 0.001 ohm
 - **Implication:** Simulates a **"Bolted Fault"** (Zero Impedance), creating the maximum possible short-circuit current for training "worst-case" scenarios.

- **Ground Resistance (R_g): 0.01 ohm**
 - **Implication:** Low impedance earth path.
- **Switching Times: External (Checked)**
 - **Implication:** Allows you to trigger the fault at exact times (e.g., $t=0.5s$) to separate "Normal" data from "Fault" data in our dataset.

Based on the detailed screenshots you provided ([image_7c9321.png](#), [image_7c95c3.png](#), [image_7c9322.png](#)), here is the deep-dive engineering breakdown of your **Main Transformer Block**.

This block is the "Heart" of your thesis. The specific numbers here are what generate the **Magnetizing Inrush Current** you are trying to detect.

7. Main Transformer

Source: [image_7c9321.png](#)

- **Winding 1 Connection: Y (Wye/Star)**
 - **Implication:** The Primary (High Voltage) side has a neutral point. This is crucial for grounding the high-voltage side of the grid.
- **Winding 2 Connection: Delta (D1)**
 - **Implication:** This is standard for Step-Down transformers.
 - **The "D1" Vector Group:** This means the Secondary voltage **lags** the Primary voltage by 30° (1 o'clock position). The differential Protection logic (or AI) *must* account for this phase shift, otherwise, it will trip incorrectly even during normal operation.
- **Core Type: Three single-phase transformers**
 - **Implication:** Modeling three separate magnetic cores (like three separate tanks) rather than one three-legged core. This prevents "magnetic coupling" between phases inside the core itself. It means the Inrush in Phase A is calculated purely based on Phase A's voltage, making the simulation cleaner and easier to analyze.
- **Nominal Power & Frequency:**
 - **Value:** 300 MVA, 50 Hz.
 - **Implication:** This defines the "Base Current" for your per-unit (pu) calculations.
 - $I_{base-primary} = 300e6 / \sqrt{3} * 230k = 753 \text{ A}$
- **Winding 1 (Primary): [230e3 0.0025 0.01]**
 - **Voltage:** 230,000 V (230 kV).
 - **Resistance:** 0.0025 pu (Very low losses, typical for large transformers).
 - **Leakage Inductance:** 0.01 pu.

- **Winding 2 (Secondary):** [11e3 0.0025 0.01]
 - **Voltage:** 11,000 V (11 kV).
- **Magnetization Resistance (Rm):** 500 pu
 - **Implication:** Represents the "Core Losses" (Eddy currents + Hysteresis). A high number (500 pu) means the core is very efficient and draws very little current when idle. Might change later
- **Magnetization Inductance (Lm):** inf
 - **Note:** This is set to infinity because we are using the Saturation Curve instead. The linear inductance is ignored.

The Saturation Characteristic

Parameter: [0 0 ; 0.005 1.2 ; 5.0 1.8] This matrix defines the B-H curve (Flux vs. Current) of the iron core. This is exactly *how* the Inrush is generated.

- **Point 1 (0, 0):** Origin. No current = No flux.
- **Point 2 (0.005, 1.2):** The "**Knee Point.**" Up to 1.2 pu Flux, the transformer is efficient. It only takes a tiny current (0.005 pu, or 0.5%) to generate full magnetic flux.
- **Point 3 (5.0, 1.8):** The "**Deep Saturation**" region. This is the Saturation point. To push the flux from 1.2 to 1.8 (a 50% increase), the current has to explode from 0.005 to 5.0 (a 1000x increase).
 - **The Result:** When we switch on the transformer, if the voltage forces the flux to go above 1.2 pu (e.g., to 1.8 pu), the current *must* jump to 5.0 pu (5x rated current). **That jump IS the Inrush Spike.**

8. CT transformers

- **Nominal Power:** 25 VA
 - **Implication:** This is the "Burden" capacity. It means the CT can drive a protection circuit that consumes up to 25 Volt-Amps without losing accuracy. If the wires are too long or the relay is too old (high impedance), the CT will fail to push the current.
- **Frequency:** 50 Hz
- **Winding 1 Parameter:** [5*1/20 0.001 0.04]
 - **Voltage (V1):** $5 \cdot 1 / 20 = 0.25 \text{ V}$. The primary of a CT is just a thick wire passing through a core. It has almost zero voltage drop. We set this low voltage to represent that it's a current-driven device, not a voltage source.
 - **Resistance (R1):** 0.001 pu. Very low (thick copper bar).
 - **Inductance (L1):** 0.04 pu.

- **Winding 2 Parameter:** [5 0.001 0.04]
 - **Voltage (V2):** 5 V.
 - By setting Primary V=0.25 and Secondary V=5, you are setting a **1:20 Voltage Ratio**.
 - **Current Ratio:** Since transformers inverse voltage and current ($V_1:V_2 = I_1:I_2$), a 1:20 voltage step-up means a **20:1 Current Step-Down**.

- **Saturation Parameter:** [0 0 ; 0.01 10 ; 1 10.5]
 - **Point 1 (0,0):** Origin.
 - **Point 2 (0.01, 10): Saturation starts.** At 0.01 pu current (very low), flux is high (10 pu).
 - **Point 3 (1, 10.5): Hard Saturation.** At 1.0 pu current (rated current), flux only increases slightly to 10.5.
 - This curve is extremely "flat" after the knee point. This means if a massive fault current flows, the CT core will saturate (fill up with flux) and stop transforming current accurately. The output will look "chopped." It's an issue we have to handle in the relay logic.

Settings for Inrush Run

1. **Secondary Breaker:** OPEN (Step time > 10s).
2. **Primary Breaker:** CLOSES (Step 0 \to 1 at t~0.05s).
3. **Internal Fault:** DISABLED (Step time > 10s).
4. **External Fault:** DISABLED (Step time > 10s).